

200Lab Education

Nga Vu - Data Mentor

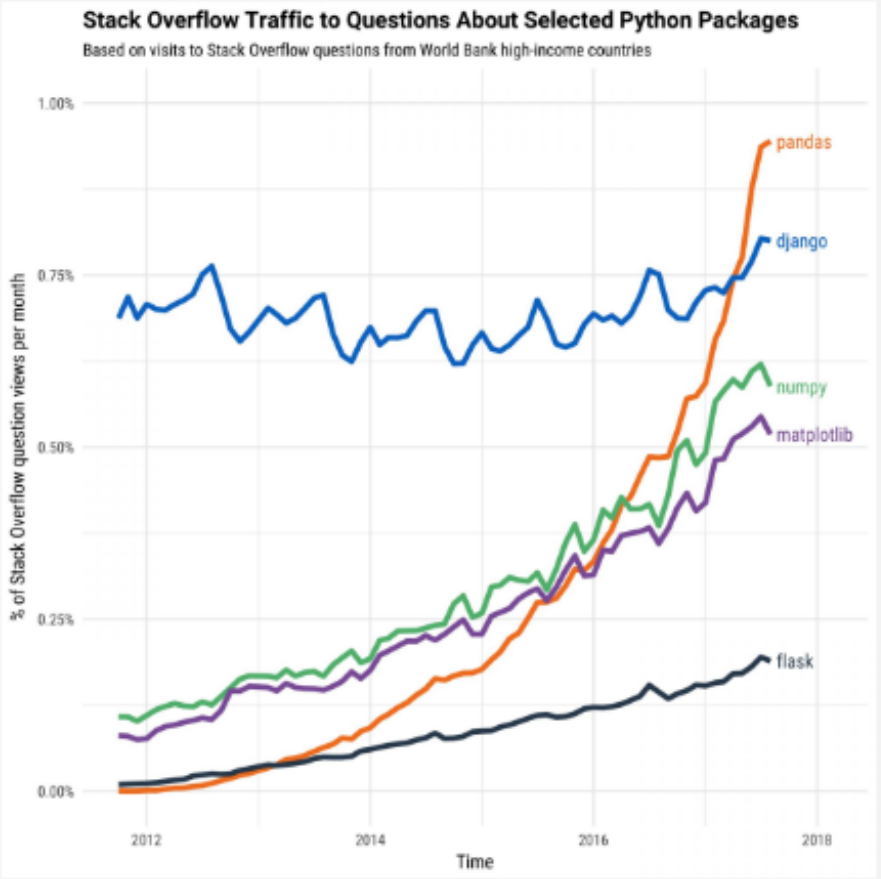
Numpy

Numerical Python

200Lab Education

What is Numpy ?

- Numpy (Numerical Python) is an open source Python library that's used in almost every field of science and engineering
- The NumPy API is used extensively in Pandas, SciPy, Matplotlib, scikit-learn, scikit-image and most other data science and scientific Python packages



matplotlib

scikit learn

NumPy

pandas

Beyond Numpy Library

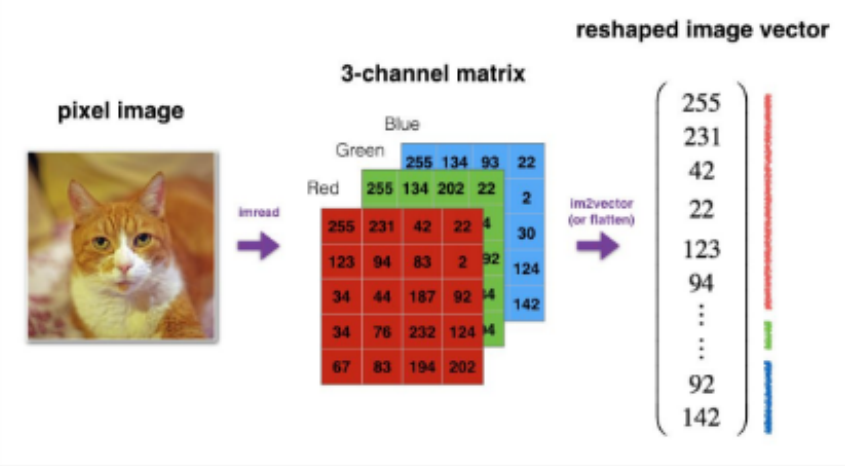
- The NumPy library contains multidimensional array and matrix data structures
- The array object in Numpy is called **ndarray** (homogeneous n-dimensional array object) : 1D, 2D, 3D, … array
- Numpy also provide methods to efficiently operate arrays:
  - Mathematical, logical, shape manipulation
  - Sorting, selecting, …
  - Linear algebra, basic statistical operations

200Lab Education

Why you need to learn Numpy ?

- NumPy arrays are faster and more compact than Python lists.
- An array consumes less memory and is convenient to use.
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data

=> Most libraries deal with data cleaning, transformation, … or even machine learning libraries use Numpy to have better performance like Pandas, TensorFlow, scikit-learn, …



Color	Red	Yellow	Green
Red	1	0	0
Red	1	0	0
Yellow	0	1	0
Green	0	0	1
Yellow	0	1	0

200Lab Education

Installing Numpy

- Google Colab (Jupyter notebook) : https://colab.research.google.com/

```
!pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.7/dist-packages (1.21.6)
```

Google already installed it for you in advance so it shows “already satisfied”

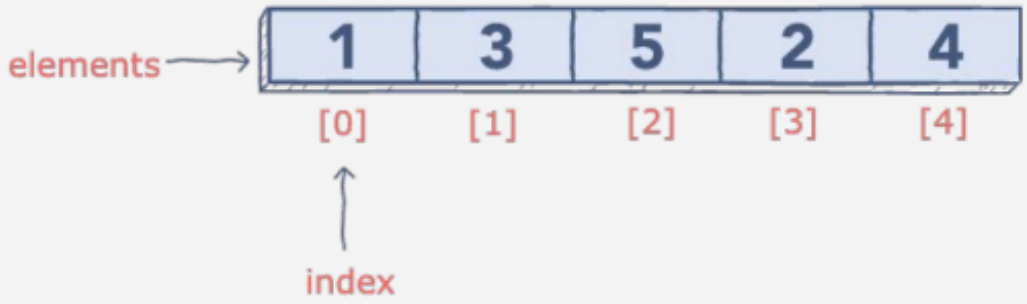
Ndarray vs Python Sequences

- Python supports six different types of sequences: strings, lists, tuples, byte sequences, byte arrays, and range objects
- A Python list can contain different data types within a single list, all of the elements in a NumPy array should be homogeneous or the same data type, and thus will be the same size in memory
- NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ndarray will create a new array and delete the original
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data.
- Scientific and mathematical Python-based packages are using NumPy arrays

200Lab Education

What is an array ?

- An array is a grid of values and it contains information about the raw data, how to locate an element, and how to interpret an element.
- It has a grid of elements that can be indexed
- We can access the elements in the array using square brackets
- Indexing in Numpy starts at 0




```
a = np.array([1, 2, 3, 4, 5, 6])
```

```
print(a[0])
```

```
1
```

200Lab Education

What is an array ?

- The elements are all of the same type (**array dtype**).
- The shape of the array giving **the size** of the array along each dimension
- The rank of the array is the number of dimensions: rank of a is 2, b is 1

```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
a.shape
```

```
(3, 4)
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
b = np.array([1, 2, 3, 4, 5, 6])
b.shape
```

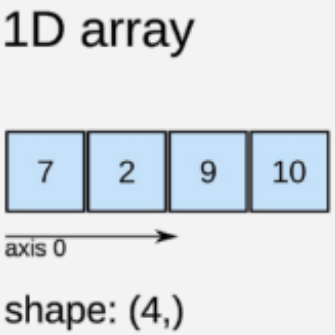
```
(6,)
```

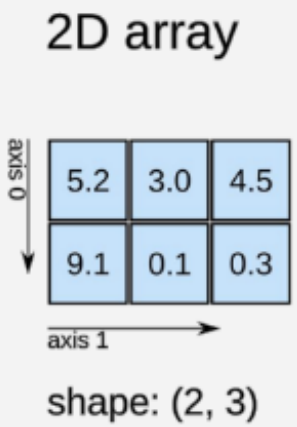
200Lab Education

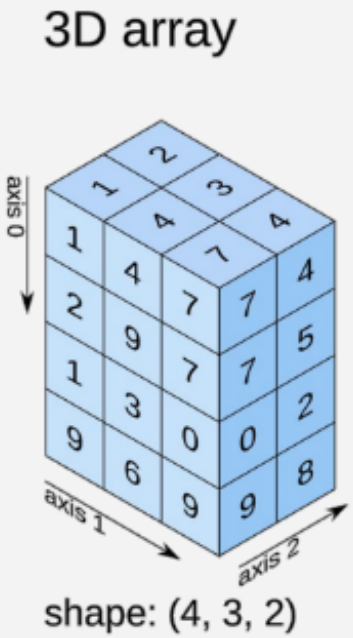
What is an array ?

- A vector is an array with a single dimension - 1D array
- A matrix refers to an array with two dimensions - 2D array
- For 3-D or higher dimensional arrays, the term **tensor** is also commonly used

TensorFlow - A tensor is a generalization of vectors and matrices to potentially higher dimensions. Internally, **TensorFlow represents tensors as n-dimensional arrays** of base datatypes





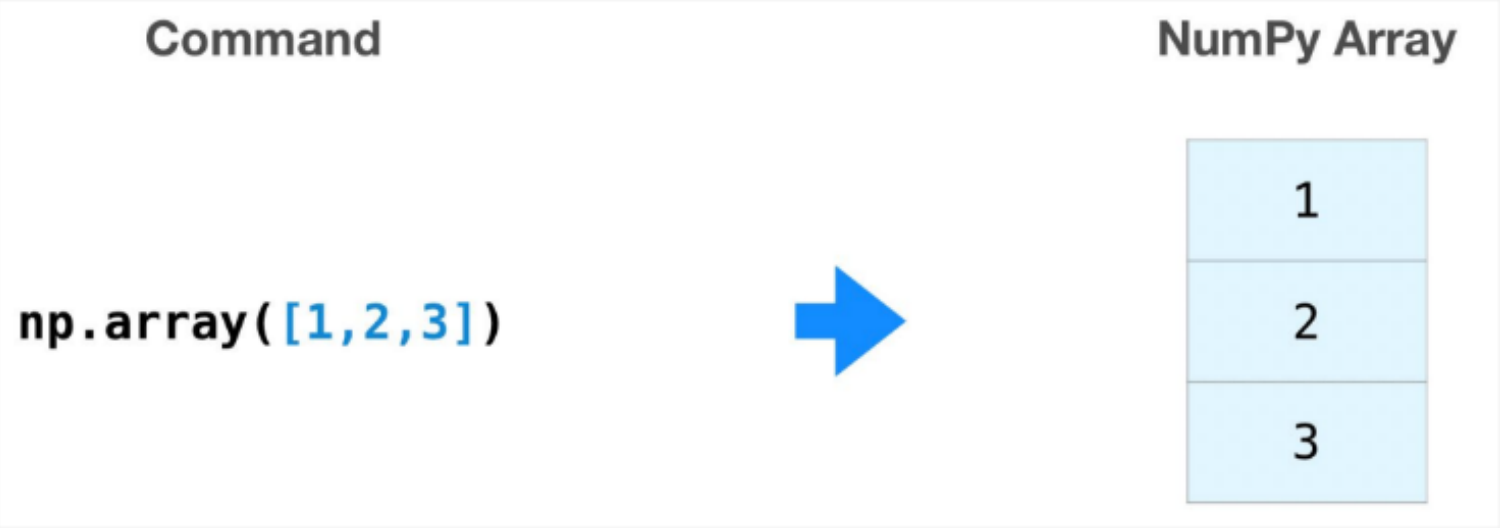


200Lab Education

How to create a basic array

To create a NumPy array, you can use the function **np.array()** and simply pass a **list** to it

```
import numpy as np
a = np.array([1, 2, 3])
```



How to create a basic array

Create an array filled with 0's

```
>>> np.zeros(2)
array([0., 0.])
```

```
a = np.zeros([3,2])
```

Create an array filled with 1's

```
>>> np.ones(2)
array([1., 1.])
```

Create an empty array

```
>>> np.empty(2)
array([ 3.14, 42.  ])
```

How to create a basic array

Create an array with a range of elements

```
>>> np.arange(4)
array([0, 1, 2, 3])
```

more complicated range

```
>>> np.arange(2, 9, 2)
array([2, 4, 6, 8])
```

You can also use **np.linspace()** to create an array with values that are spaced linearly in a specified interval

```
>>> np.linspace(0, 10, num=5)
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

How to create a basic array

While the default data type is floating point (**np.float64**), you can explicitly specify which data type you want using the **dtype** keyword

```
>>> x = np.ones(2, dtype=np.int64)
>>> x
array([1, 1])
```

Important attributes of an array

- **ndarray.ndim**: number of axes, or dimensions, of the array.
- **ndarray.size**: total number of elements of the array.
- **ndarray.shape**: number of elements stored along each dimension of the array. If you have a 2-D array with 2 rows and 3 columns, the shape of your array is (2, 3)

```
array_example = np.array([[[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0 ,1 ,2, 3],
                           [4, 5, 6, 7]]])
```

```
>>> array_example.ndim
3
```

```
>>> array_example.size
24
```

```
>>> array_example.shape
(3, 2, 4)
```

200Lab Education

Important attributes of an array

- **ndarray.dtype**: describing the type of the elements in the array
- **ndarray.itemsize**: the size in bytes of each element of the array

```
array_example = np.array([[[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0, 1, 2, 3],
                           [4, 5, 6, 7]],

                          [[0 ,1 ,2, 3],
                           [4, 5, 6, 7]]])
```

```
array_example.dtype
```

```
dtype('int64')
```

```
array_example.itemsize
```

```
8
```

Sorting elements

Sort the numbers in ascending order

```
arr = np.array([2, 1, 5, 3, 7, 4, 6, 8])
```

```
>>> np.sort(arr)
array([1, 2, 3, 4, 5, 6, 7, 8])
```

Reverse order ?

```
array = np.array([7, 5, 3, 2, 6, 1, 4])

# Sort in ascending order
sorted_array = np.sort(array)
# [1, 2, 3, 4, 5, 6, 7]

# Reverse the sorted array
reverse_array = sorted_array[::-1]
# [7, 6, 5, 4, 3, 2, 1]
```

Sorting elements

np.argsort(), which is an indirect sort along a specified axis

```
>>> x = np.array([3, 1, 2])
>>> np.argsort(x)
array([1, 2, 0])
```

```
objects = ['Dog','Cat','Bird']
```

np.lexsort(), which is an indirect stable sort on multiple keys

```
>>> a = [1,5,1,4,3,4,4] # First column
>>> b = [9,4,0,4,0,2,1] # Second column
>>> ind = np.lexsort((b,a)) # Sort by a, then by b
>>> ind
array([2, 0, 4, 6, 5, 3, 1])
```

200Lab Education

Add & Remove elements

Add elements: np.append(), np.insert()

```
a = np.array([[1, 1], [2, 2], [3, 3]])
np.insert(a, 1, 5)
```

```
array([1, 5, 1, 2, 2, 3, 3])
```

```
print(np.insert(a, 1, 5,axis=0))
# same as np.insert(a, 1, [5,5],axis=0)
```

```
[[1 1]
 [5 5]
 [2 2]
 [3 3]]
```

```
np.insert(a, 1, 5,axis=1)
```

```
array([[1, 5, 1],
       [2, 5, 2],
       [3, 5, 3]])
```

```
a = np.array([[1, 1], [2, 2], [3, 3]])
np.insert(a, [1], [[1],[2],[3]], axis=1)
```

```
array([[1, 1, 1],
       [2, 2, 2],
       [3, 3, 3]])
```

```
a = np.array([[1, 1], [2, 2], [3, 3]])
np.insert(a, [1,2], 5,axis=0)
```

```
array([[1, 1],
       [5, 5],
       [2, 2],
       [5, 5],
       [3, 3]])
```

Add & Remove elements

Add elements: **np.append(), np.insert()**

```
>>> np.append([1, 2, 3], [[4, 5, 6], [7, 8, 9]])
array([1, 2, 3, ..., 7, 8, 9])
```

```
>>> np.append([[1, 2, 3], [4, 5, 6]], [[7, 8, 9]], axis=0)
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> np.append([[1, 2, 3], [4, 5, 6]], [7, 8, 9], axis=0)
Traceback (most recent call last):
```

Add & Remove elements

Remove elements: np.delete()

```
>>> arr = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
>>> arr
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
>>> np.delete(arr, 1, 0)
array([[ 1,  2,  3,  4],
       [ 9, 10, 11, 12]])
```

```
np.delete(arr,1,axis=1)
```

```
array([[ 1,  3,  4],
       [ 5,  7,  8],
       [ 9, 11, 12]])
```

```
>>> np.delete(arr, [1,3,5], None)
array([ 1,  3,  5,  7,  8,  9, 10, 11, 12])
```

Shape Manipulation

array.ravel(): Return a contiguous flattened array

```
array = np.random.randint(10, size=(3,4))
array
```

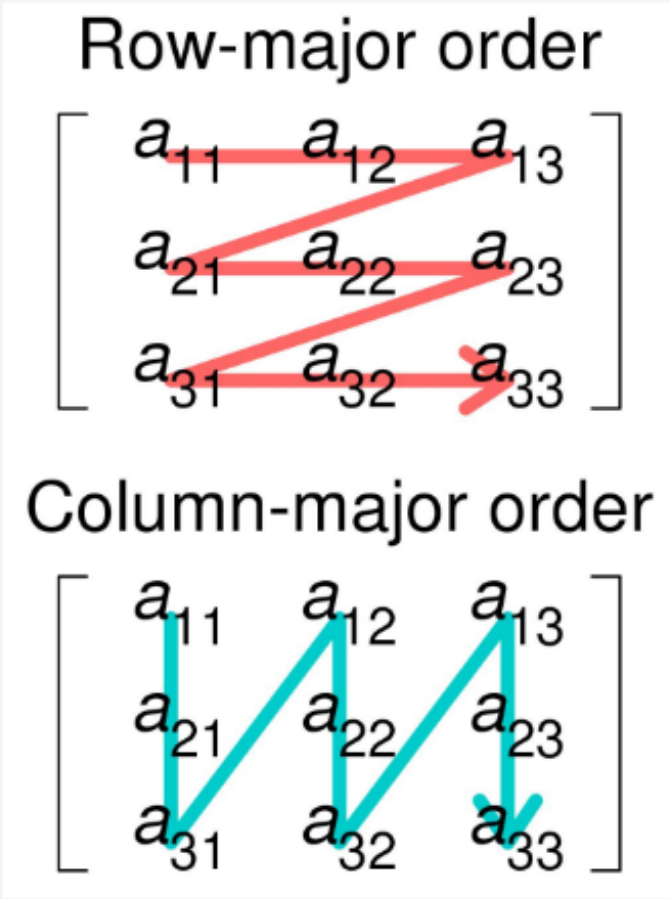
```
array([[2, 5, 5, 3],
       [1, 5, 6, 6],
       [5, 5, 8, 1]])
```

```
array.ravel()
```

```
array([2, 5, 5, 3, 1, 5, 6, 6, 5, 5, 8, 1])
```

```
array.ravel(order='F')
```

```
array([2, 1, 5, 5, 5, 5, 5, 6, 8, 3, 6, 1])
```



Shape Manipulation

array.reshape(): Gives a new shape to an array without changing its data

```
array.reshape(6,2)
```

```
array([[2, 5],
       [5, 3],
       [1, 5],
       [6, 6],
       [5, 5],
       [8, 1]])
```

```
array.ravel()
```

```
array([2, 5, 5, 3, 1, 5, 6, 6, 5, 5, 8, 1])
```

```
array.ravel(order='F')
```

```
array([2, 1, 5, 5, 5, 5, 5, 6, 8, 3, 6, 1])
```

```
array.reshape((6,2),order='F')
```

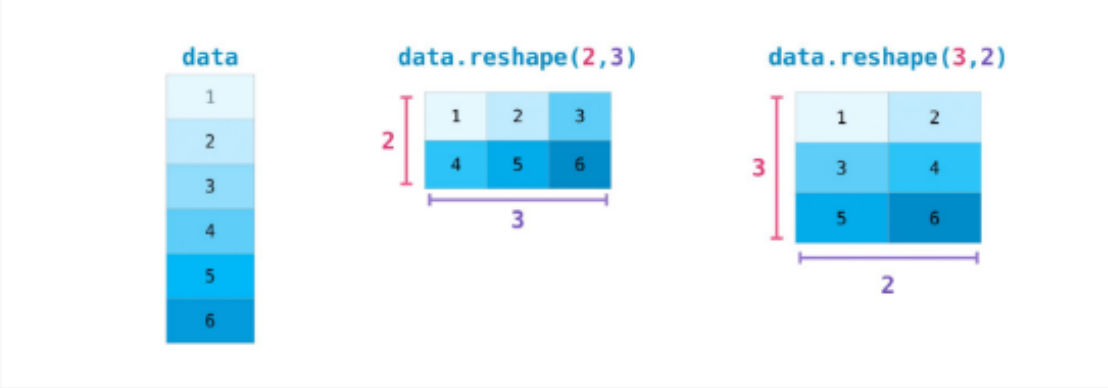
```
array([[2, 5],
       [1, 6],
       [5, 8],
       [5, 3],
       [5, 6],
       [5, 1]])
```

200Lab Education

Shape Manipulation

array.transpose(): reverse or change the axes of an array according to the values you specify





Note that the following three commands
all return a modified array,
but do not change the original array

```
array.T
```

```
array([[2, 1, 5],
       [5, 5, 5],
       [5, 6, 8],
       [3, 6, 1]])
```

Indexing, Slicing and Iterating

One-dimensional arrays can be indexed, sliced and iterated over, much like lists and other Python sequences

```
>>> a = np.arange(10)**3
>>> a
array([  0,   1,   8,  27,  64, 125, 216, 343, 512, 729])
>>> a[2]
8
>>> a[2:5]
array([ 8, 27, 64])
```

```
a[:6:2]
```

```
[0, 8, 64]
```

```
>>> # equivalent to a[0:6:2] = 1000;
>>> # from start to position 6, exclusive, set every 2nd element to 1000
>>> a[:6:2] = 1000
>>> a
array([1000,    1, 1000,   27, 1000,  125,  216,  343,  512,  729])
>>> a[::-1]  # reversed a
array([ 729,  512,  343,  216,  125, 1000,   27, 1000,    1, 1000])
```

```
>>> for i in a:
...     print(i**(1 / 3.))
...
9.999999999999998
1.0
```

Indexing, Slicing and Iterating

Multidimensional arrays
can have one index per axis

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> b[2, 3]
23
>>> b[0:5, 1]  # each row in the second column of b
array([ 1, 11, 21, 31, 41])
>>> b[:, 1]    # equivalent to the previous example
array([ 1, 11, 21, 31, 41])
>>> b[1:3, :]  # each column in the second and third row of b
array([[10, 11, 12, 13],
       [20, 21, 22, 23]])
```

The missing indices are considered complete slices

```
>>> b[-1]   # the last row. Equivalent to b[-1, :]
array([40, 41, 42, 43])
```

Array Concatenation and Splitting

Concatenation of arrays

np.concatenate()

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])
np.concatenate([x, y])
```

```
array([1, 2, 3, 3, 2, 1])
```

```
z = [99, 99, 99]
print(np.concatenate([x, y, z]))
```

```
[ 1  2  3  3  2  1 99 99 99]
```

```
grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
```

```
# concatenate along the first axis
np.concatenate([grid, grid])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the second axis (zero-indexed)
np.concatenate([grid, grid], axis=1)
```

```
array([[1, 2, 3, 1, 2, 3],
       [4, 5, 6, 4, 5, 6]])
```

Array Concatenation and Splitting

Concatenation of arrays

np.vstack(), and np.hstack()

```
# horizontally stack the arrays
y = np.array([[99],
              [99]])
np.hstack([grid, y])
```

```
array([[ 9,  8,  7, 99],
       [ 6,  5,  4, 99]])
```

```
x = np.array([1, 2, 3])
grid = np.array([[9, 8, 7],
                 [6, 5, 4]])

# vertically stack the arrays
np.vstack([x, grid])
```

```
array([[1, 2, 3],
       [9, 8, 7],
       [6, 5, 4]])
```

Array Concatenation and Splitting

Splitting of arrays

np.split()

```
>>> x = np.arange(8.0)
>>> np.split(x, [3, 5, 6, 10])
[array([0.,  1.,  2.]),
 array([3.,  4.]),
 array([5.]),
 array([6.,  7.]),
 array([], dtype=float64)]
```

```
>>> x = np.arange(9.0)
>>> np.split(x, 3)
[array([0.,  1.,  2.]), array([3.,  4.,  5.]), array([6.,  7.,  8.])]
```

Array Concatenation and Splitting

Splitting of arrays

np.hsplit()

```
>>> np.hsplit(x, np.array([3, 6]))
[array([[ 0.,  1.,  2.],
       [ 4.,  5.,  6.],
       [ 8.,  9., 10.],
       [12., 13., 14.]]),
 array([[ 3.],
       [ 7.],
       [11.],
       [15.]]),
 array([], shape=(4, 0), dtype=float64)]
```

```
>>> x = np.arange(16.0).reshape(4, 4)
>>> x
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])
>>> np.hsplit(x, 2)
[array([[ 0.,  1.],
       [ 4.,  5.],
       [ 8.,  9.],
       [12., 13.]]),
 array([[ 2.,  3.],
       [ 6.,  7.],
       [10., 11.],
       [14., 15.]])]
```

Array Concatenation and Splitting

Splitting of arrays

np.vsplit()

```
>>> x = np.arange(16.0).reshape(4, 4)
>>> x
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])
>>> np.vsplit(x, 2)
[array([[0., 1., 2., 3.],
       [4., 5., 6., 7.]]), array([[ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])]
>>> np.vsplit(x, np.array([3, 6]))
[array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.]]), array([[12., 13., 14., 15.]]),
```

200Lab Education

Numpy Ufuncs

- A universal function (or ufunc for short) is a function that operates on ndarrays in an element-by-element fashion
- Quickly execute repeated operations on values in NumPy arrays
- Ufuncs exist in two flavors: unary ufuncs, which operate on a single input, and binary ufuncs, which operate on two inputs

```
import numpy as np
np.random.seed(0)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

values = np.random.randint(1, 10, size=5)
compute_reciprocals(values)
```

```
array([ 0.16666667,  1.        ,  0.25      ,  0.25      ,  0.125     ])
```

```
print(compute_reciprocals(values))
print(1.0 / values)
```

```
[ 0.16666667  1.          0.25        0.25        0.125     ]
[ 0.16666667  1.          0.25        0.25        0.125     ]
```

```
np.arange(5) / np.arange(1, 6)
```

```
array([ 0.        ,  0.5       ,  0.66666667,  0.75      ,  0.8       ])
```

Numpy Ufuncs

Array arithmetic

Standard operators: addition (+), subtraction (-), multiplication (*), and division (/)

Operator	Equivalent ufunc	Description
`+`	`np.add`	Addition (e.g., `1 + 1 = 2`)
`-`	`np.subtract`	Subtraction (e.g., `3 - 2 = 1`)
`-`	`np.negative`	Unary negation (e.g., `-2`)
`*`	`np.multiply`	Multiplication (e.g., `2 * 3 = 6`)
`/`	`np.divide`	Division (e.g., `3 / 2 = 1.5`)
`//`	`np.floor_divide`	Floor division (e.g., `3 // 2 = 1`)
`**`	`np.power`	Exponentiation (e.g., `2 ** 3 = 8`)
`%`	`np.mod`	Modulus/remainder (e.g., `9 % 4 = 1`)

```
x = np.arange(4)
print("x     =", x)
print("x + 5 =", x + 5)
print("x - 5 =", x - 5)
print("x * 2 =", x * 2)
print("x / 2 =", x / 2)
print("x // 2 =", x // 2)  # floor division
```

```
x     = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [ 0.   0.5  1.   1.5]
x // 2 = [0 0 1 1]
```

```
print("-x     = ", -x)
print("x ** 2 = ", x ** 2)
print("x % 2  = ", x % 2)
```

```
-x     =  [ 0 -1 -2 -3]
x ** 2 =  [0 1 4 9]
x % 2  =  [0 1 0 1]
```

200Lab Education

Numpy Ufuncs

Aggregation

Function Name	NaN-safe Version	Description
`np.sum`	`np.nansum`	Compute sum of elements
`np.prod`	`np.nanprod`	Compute product of elements
`np.mean`	`np.nanmean`	Compute mean of elements
`np.std`	`np.nanstd`	Compute standard deviation
`np.var`	`np.nanvar`	Compute variance
`np.min`	`np.nanmin`	Find minimum value
`np.max`	`np.nanmax`	Find maximum value
`np.argmin`	`np.nanargmin`	Find index of minimum value
`np.argmax`	`np.nanargmax`	Find index of maximum value
`np.median`	`np.nanmedian`	Compute median of elements
`np.percentile`	`np.nanpercentile`	Compute rank-based statistics of elements
`np.any`	N/A	Evaluate whether any elements are true
`np.all`	N/A	Evaluate whether all elements are true

```
np.min(big_array), np.max(big_array)
```

```
(1.1717128136634614e-06, 0.9999976784968716)
```

```
print(big_array.min(), big_array.max(), big_array.sum())
```

```
1.17171281366e-06 0.999997678497 499911.628197
```

```
M = np.random.random((3, 4))
print(M)
```

```
[[ 0.8967576   0.03783739  0.75952519  0.06682827]
 [ 0.8354065   0.99196818  0.19544769  0.43447084]
 [ 0.66859307  0.15038721  0.37911423  0.6687194 ]]
```

```
M.sum()
```

```
6.0850555667307118
```

```
M.min(axis=0)
```

```
array([ 0.66859307,  0.03783739,  0.19544769,  0.06682827])
```

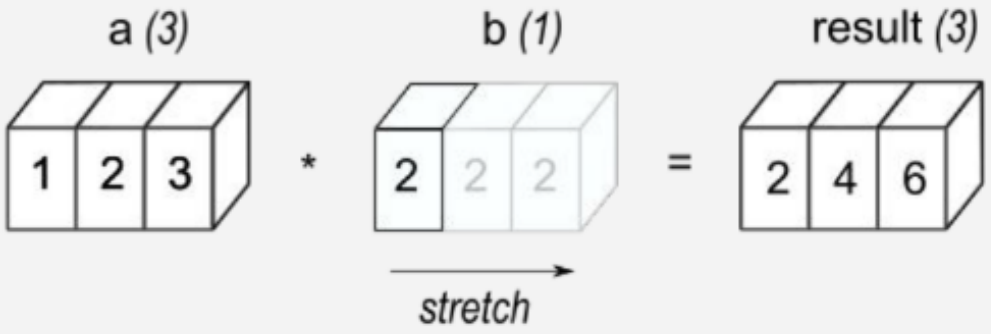
200Lab Education

Broadcasting

Broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations

```
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = np.array([2.0, 2.0, 2.0])
>>> a * b
array([2.,  4.,  6.])
```

```
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = 2.0
>>> a * b
array([2.,  4.,  6.])
```



200Lab Education

Broadcasting

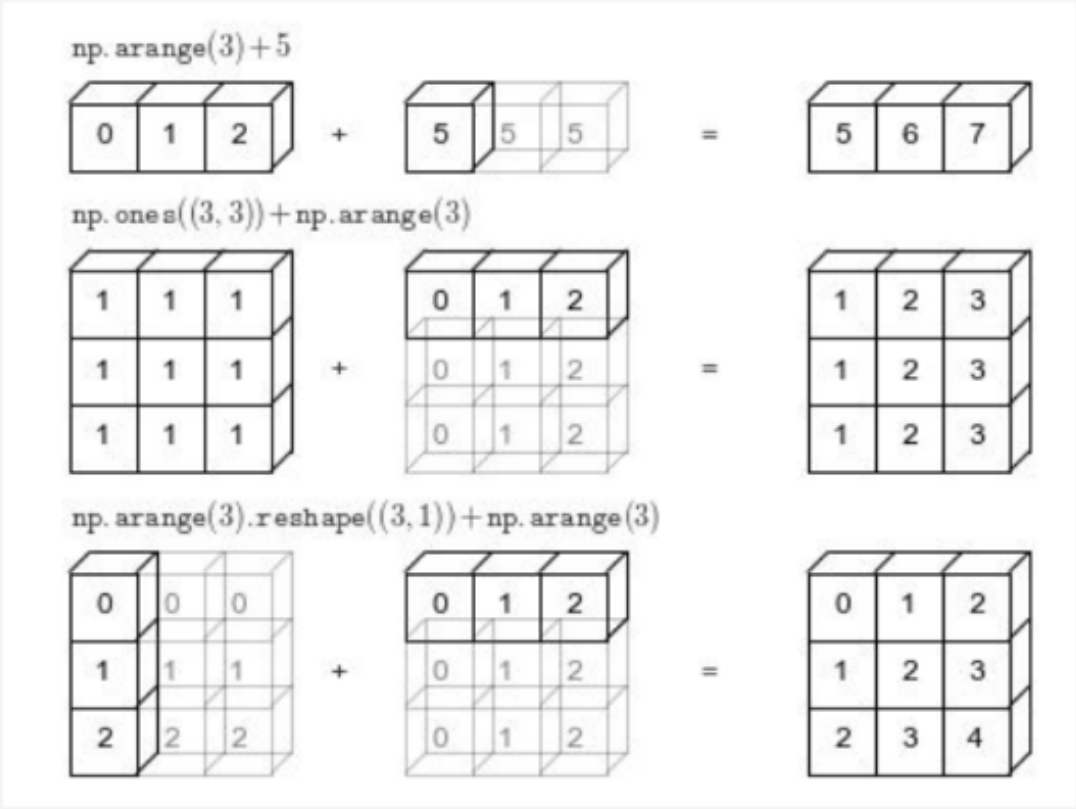
Broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations

```
print(a)
print(b)
```

```
[0 1 2]
[[0]
 [1]
 [2]]
```

```
: a + b
```

```
: array([[0, 1, 2],
         [1, 2, 3],
         [2, 3, 4]])
```



Filter Elements

- **Boolean Masking**: also called boolean indexing, is a feature in NumPy that allows for the filtering of values in numpy arrays
- To create boolean index list, we have 2 common ways:
  - create a boolean array based on a logical condition (>,<,>=, ...)
  - np.where()

```
a = np.array([41, 42, 43, 44])
b = [True, False, True, False]
print(a[b])
```

```
[41 43]
```

Filter Elements

```
a = np.array([41, 42, 43, 44])
b = [True, False, True, False]
print(a[b])
```

```
[41 43]
```

```
a<43
```

```
array([ True,  True, False, False])
```

```
a[a<43]
```

```
array([41, 42])
```

```
a[(a<43) & (a>10)]
```

```
array([41, 42])
```

```
a = np.array([41, 42, 43, 44])
b = np.array([40, 45, 50, 40])
np.where(a>b,a,b)
```

```
array([41, 45, 50, 44])
```

```
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.where(a < 5, a, 10*a)
array([ 0,  1,  2,  3,  4, 50, 60, 70, 80, 90])
```

```
>>> np.where([[True, False], [True, True]],
...           [[1, 2], [3, 4]],
...           [[9, 8], [7, 6]])
array([[1, 8],
       [3, 4]])
```

```
a= np.array([[41,42],[43,44]])
b = np.array([[40,45],[50,40]])
np.where(a>b,a,b)
```

```
array([[41, 45],
       [50, 44]])
```